National Centre for Atmospheric Science
NATURAL ENVIRONMENT RESEARCH COUNCIL

Climate and Forecasting Aggregation (CFA) file preparation for the CANARI climate model ensemble

Jonny Williams*, David Hassell, Bryan Lawrence
NCAS-CMS

July 9, 2026

Contents

*jonny.williams@ncas.ac.uk

0 Project structure

Figure 1 and Listing 1 show the project structure and the code to generate it respectively.

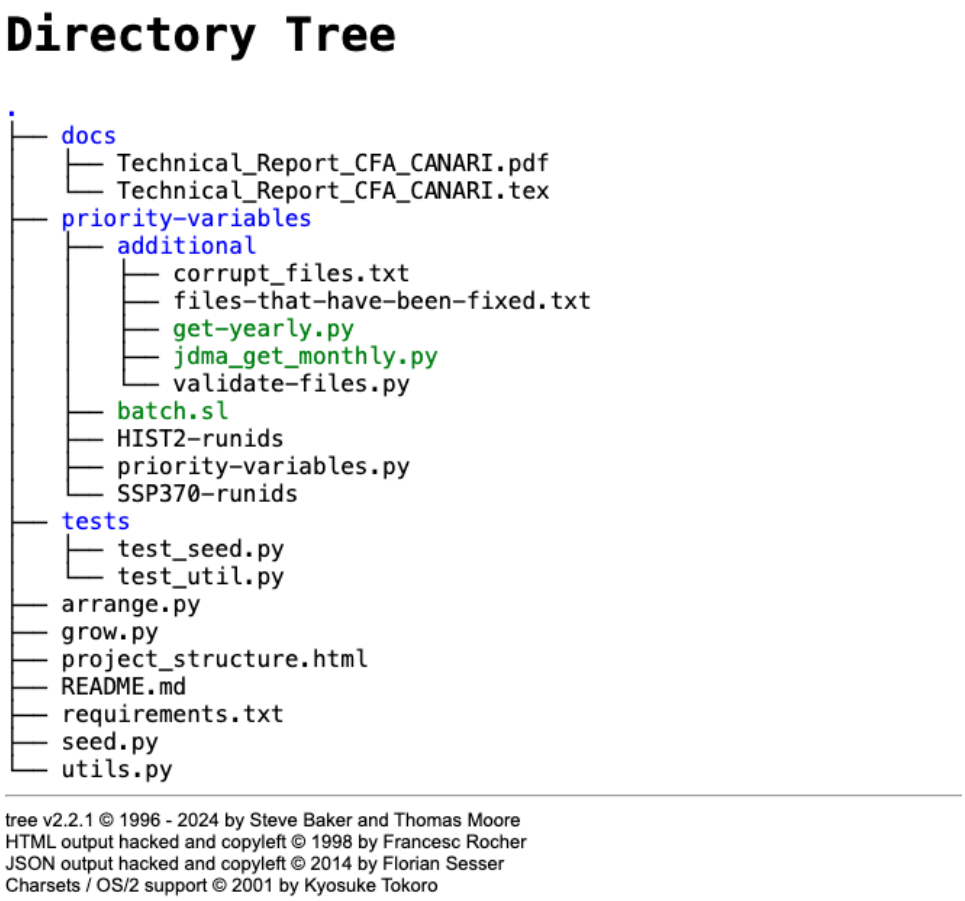

Figure 1

Listing 1: `tree...` command run in base of repository.

```
tree -F --dirsfirst --noreport -C -I 'CFA-files|HIST2|SSP370|u-*|_*|*.cfa|plantUML|
    figures|bib*|*png|go|*.aux|*.bbl|*.blg|*.fdb_latexmk|*.fls|*.log|*.out|*.synctex.
    gz|*.toc' -H . -o project_structure.html
```

1 Introduction

This document details the processes by which data from the CANARI project – large ensembles of simulations using the HadGEM3-GC3.1 climate model [Williams et al., 2018] – has been aggregated after its ingestion into the Joint-storage Data Migration App (JDMA) archive[1]. The JDMA archive is hosted on the JASMIN platform hosted by the Centre for Environmental Data Analysis, CEDA[2].

The CANARI ensemble consists of two, 40-member ensembles:

1. Historical simulations running from 1850-2015.
2. SSP3-7.0 future scenarios, 2015-2100.

SSP stands for Shared Socioeconomic Pathway. These are a family of potential future greenhouse gas and land-use pathways which are standardised and extensively used in the climate science community, more information can be found in IPCC [2023].

Within the respective ensembles, each member differs only in its initial conditions, which are generated using a hybrid 'macro-micro' initialisation method [Schiemann et al., 2026].

Each simulation generates data in three month 'batches' and the format of the batches' contents are described as, e.g., `runid/cyclepoint`, where `runid` takes the form `u-ab123` and `cyclepoint` is in the ISO-8601 'datetime' standard, e.g. `19500101T0000Z` [International Organization for Standardization, 2019]. Each batch is then given an integer index which describes its 'location' on the JDMA archive. These values are given on the NCAS-CMS GitHub pages[3] and an example is given in Figure 2. The data is held in monthly files and so each batch contains three sets of data in a common format.

JDMA batch numbers

```
sci1.jasmin.ac.uk:reinhard$ jdma batch -f workspace -w canari | grep u-cv575 | cat -n
     1      3203 reinhard         canari           u-cv575/19500101 elastictape      2023-03-22 15:44  ON_STORAGE
     2      3230 reinhard         canari           u-cv575/19500401 elastictape      2023-03-23 15:44  ON_STORAGE
     3      3231 reinhard         canari           u-cv575/19500701 elastictape      2023-03-23 15:54  ON_STORAGE
     4      3232 reinhard         canari           u-cv575/19501001 elastictape      2023-03-23 16:05  ON_STORAGE
     5      3240 reinhard         canari           u-cv575/19510101 elastictape      2023-03-23 23:49  ON_STORAGE
     6      3255 reinhard         canari           u-cv575/19510401 elastictape      2023-03-24 03:10  ON_STORAGE
     7      3263 reinhard         canari           u-cv575/19510701 elastictape      2023-03-24 08:51  ON_STORAGE
     8      3270 reinhard         canari           u-cv575/19511001 elastictape      2023-03-24 11:42  ON_STORAGE
     9      3279 reinhard         canari           u-cv575/19520101 elastictape      2023-03-24 16:34  ON_STORAGE
    10      3304 reinhard         canari           u-cv575/19520401 elastictape      2023-03-24 20:04  ON_STORAGE
    11      3310 reinhard         canari           u-cv575/19520701 elastictape      2023-03-24 22:45  ON_STORAGE
    12      3400 reinhard         canari           u-cv575/19521001 elastictape      2023-03-27 08:17  ON_STORAGE
    13      3401 reinhard         canari           u-cv575/19530101 elastictape      2023-03-27 08:22  ON_STORAGE
    14      3402 reinhard         canari           u-cv575/19530401 elastictape      2023-03-27 08:27  ON_STORAGE
    15      3403 reinhard         canari           u-cv575/19530701 elastictape      2023-03-27 08:32  ON_STORAGE
    16      3404 reinhard         canari           u-cv575/19531001 elastictape      2023-03-27 08:42  ON_STORAGE
    17      3405 reinhard         canari           u-cv575/19540101 elastictape      2023-03-27 08:48  ON_STORAGE
```

Figure 2: Example of JDMA batch indexing from the CANARI project pages.

2 Workflow

The workflow discussed here assumes that the diagnostic content of each simulation is the same. In almost every respect this is the case, except for four variables in the atmospheric output in the historical simulations; this will be discussed in § 2.4.1. This assumption significantly reduces the time taken to generate the 'seed' files described below which make up the vast majority of

[1] https://help.jasmin.ac.uk/docs/short-term-project-storage/jdma/
[2] https://www.ceda.ac.uk
[3] https://ncas-cms.github.io/canari

ID	Historical	SSP3-7.0	ID	Historical	SSP3-7.0
1	u-cv575 *	u-de814	21	u-da179	u-df933
2	u-cv625 *	u-de436	22	u-da190	u-df934
3	u-cw345 *	u-de724	23	u-da191	u-df935
4	u-cw356 *	u-de815	24	u-da192	u-df936
5	u-cv827 *	u-df220	25	u-da193	u-df937
6	u-cv976 *	u-de830	26	u-db291	u-dh412
7	u-cz547 *	u-de831	27	u-db301	u-dh413
8	u-cy436 *	u-de832	28	u-db303	u-dh415
9	u-cw342 *	u-de850	29	u-db304	u-dh416
10	u-cw343 *	u-de851	30	u-db305	u-dh417
11	u-cy375 *	u-de934	31	u-cz475	u-di511
12	u-cy376 *	u-de937	32	u-cz568	u-di512
13	u-cy537	u-de938	33	u-cz647	u-di513
14	u-cy811	u-de939	34	u-cz648	u-di514
15	u-cy866	u-de940	35	u-cz649	u-di515
16	u-cy873	u-df299	36	u-dd436	u-di703
17	u-cy877	u-df300	37	u-dd438	u-di704
18	u-cy879	u-df301	38	u-dd439	u-di705
19	u-cy880	u-df302	39	u-dd441	u-di706
20	u-cy881	u-df303	40	u-dd442	u-di707

Table 1: Ensemble Member RunIDs for Control and SSP3-7.0 scenarios. The simulations marked with asterisks have a slightly different output variable set and the ensemble members used as 'seeds' are highlighted; see § 2.4.

the processing time. Table 1 shows the simulation identifiers for the historical and SSP3-7.0 simulations. Their format is standard for climate models run with the Rose and Cylc [Oliver et al., 2018] workflow tools. Rose is 'a framework for managing and running meteorological suites'[4] and Cylc is 'a general purpose workflow engine with a particular gift for cycling'[5].

2.1 Aggregation with `cf` Python

CF aggregation is a new feature in the CF conventions, having been available since December 2025 [Eaton et al., 2025]. Their usage in this particular application is in the representation of a large database of climate model data – $\mathcal{O}$(PB) – in a format which is portable and easily reproducible – $\mathcal{O}$(100MB) or less. The workflow must also be achievable in a timescale which enables easy reruns and big fixes, $\mathcal{O}$(1 day) or less.

2.2 Workflow diagram

After choosing the scenario and seed simulation ID, the workflow is made up of three stages; seed, grow and arrange. This is illustrated in Figure 3. The individual algorithms for the seed, grow and arrange steps are shown in Figure 5, 6 and 8 respectively.

[4] https://www.metoffice.gov.uk/research/approach/modelling-systems/rose
[5] https://cylc.github.io

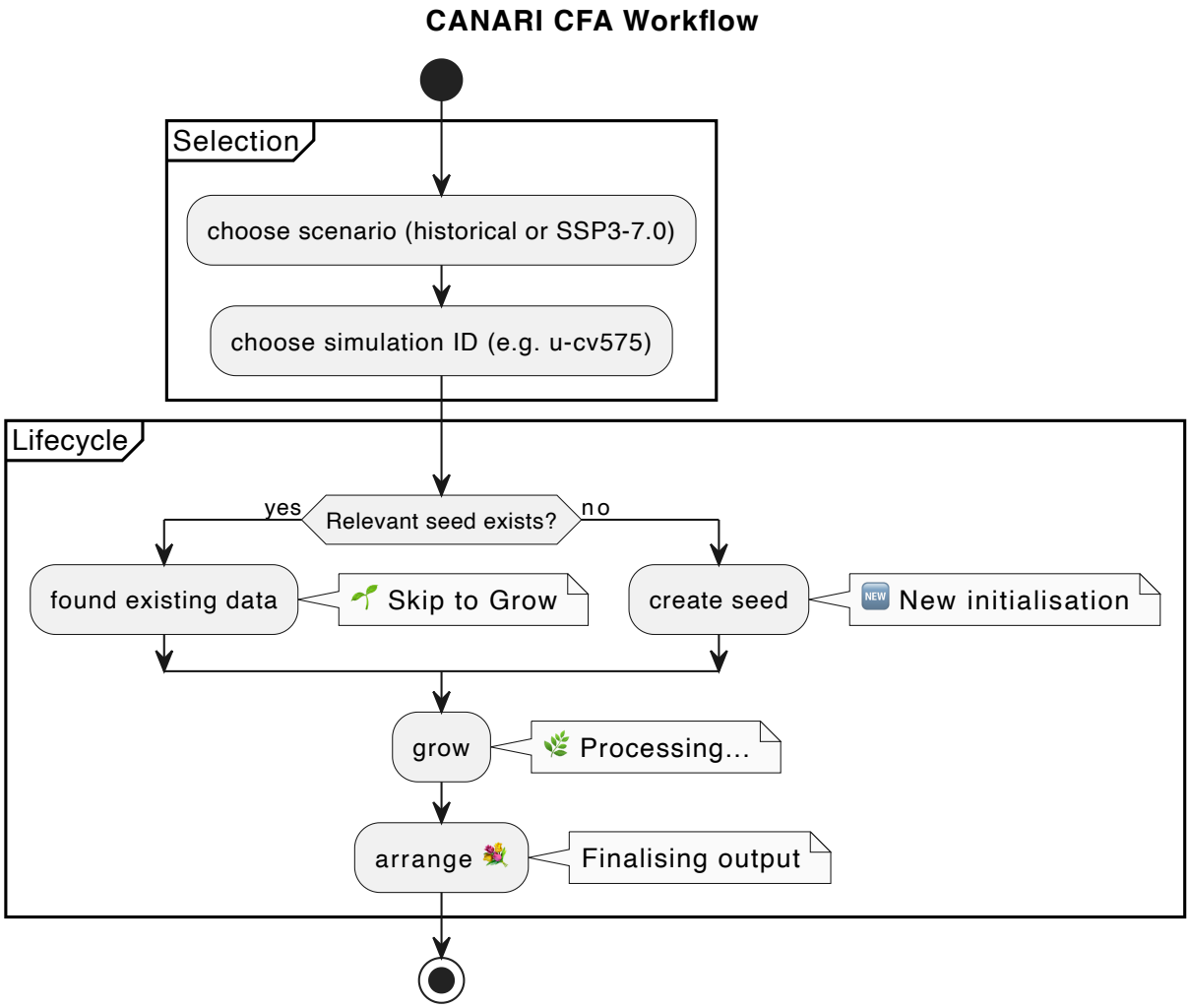

Figure 3: Workflow diagram of the whole process of generating the CFA files, including choosing the scenario and simulation ID. Figure prepared using `PlantUML`.

```
fragment_unique_values = "195001" ;

m01s00i002 = _ ;

fragment_map_1 =
 {120},
 {85},
 {324},
 {432} ;

fragment_uris =
 "cv575a_1_6hr_u_pt_cordex__195001-195001.nc" ;
```

Figure 4: `fragment_unique_values` and `fragment_unique_values` variables in the historical simulations' seed CFA file.

2.3 Seed

The seed stage requires one complete month (only) of data on locally accessible disk. It is this month which is interrogated by the Python script `seed.py` and generates a seed file which is then 'grown' – § 2.4 – and 'arranged' – § 2.5 – to give the final CFA file for the complete run length of the simulation (see page 4).

For example, the files which are used in the creation of the seed file for the historical ensemble are of the form `cv575{a,o,i}_1*195001*.nc`, where `{a,o,i}` uses the standard Linux wildcard syntax for 'a or o or i' to refer to the atmosphere, ocean and sea ice datasets respectively. Going forward we use the term 'realm' to refer to this distinction. This in turn assumes a consistent naming convention of the post processed ensemble data, and we assume this to be the case throughout the ensembles.

Of primary importance for the understanding of later sections of this document are the variables which describe the following:

- File names
  - These are called `fragment_uris` in the language of the CF conventions[6] where `uris` are Uniform Resource Identifiers.
- JDMA batch numbers
  - These are called `fragment_unique_values` in the CF conventions.

Figure 4 shows a screenshot of a section of the `ncdump` output from the resulting seed CFA file. The `fragment_unique_values` and `fragment_uris` variables are highlighted.

Note that in the seed file, the `fragment_unique_values` string is set to the value of `year+month` strings in the `seed.py` file where `01` is January etc. This convention is also used in the `grow.py` described in § 2.4.

[6] https://cfconventions.org

The seed Python file also removes the NetCDF `name` attribute, which by default points to the original creation location on disk when the simulation was first run and is hence irrelevant – and potentially misleading – once the data has undergone any processing. The `seed.py` file must be run separately for each realm as is the case for the 'grow' and 'arrange' steps too.

Finally, Figure 5 shows a visual representation of the algorithm used in `seed.py`.

2.4 Grow

Figure 6 shows the `PlantUML` visualisation of the `grow.py` script. The majority of the code in this step of the overall workflow is concerned with the formatting of the date-time strings which, as part of the filenames – i.e. `fragment_uris` – are grown, month by month, up to the end of the simulation's runtime (historical, 65 years; SSP3-7.0 85 years).

Figure 7 shows the first 14 months for the `fragment_uris` variable[7]. The equivalent `fragment_unique_values` variables are simply a reproduction – to the same size as the `fragment_uris` – of that from the seeding step, i.e., placeholders of the form `195001` and `201501` for the historical and SSP3-7.0 ensembles respectively. The `arrange.py` edits these 'dummy' values and is described in § 2.5.

The growth step is only required to be run for the 'seed' simulations, i.e. the highlighted ones in Table 1. The arrangement step creates the final required files for the entire ensemble.

2.4.1 Note on historical diagnostic sets

For the historical ensemble, members 1-13 have slightly different variable sets[8] to the remainder. Therefore, members 2-12 were seeded from member 1, `u-dv575`, and 14-40 were seeded from member 13, `u-cy537`. All SSP3-7.0 members were seeded from its first member, `u-de814`.

2.5 Arrange

2.5.1 JDMA batch IDs

Figure 2.5 shows the algorithm for the 'arrangement' step.

When the CANARI ensemble was run, the JDMA batch numbers associated with each three month cycle were manually documented – see page 1 – and these values are used as 'lookup tables' to find the relevant batch number from the datetime string in the `fragment_uri` values. An important step here is to convert the monthly – `195001, 195002` – datetime strings into the correct three-monthly cycle points. The need for this is illustrated in Figure 2 where the batches are names `195001`, `195004`, etc. This is the 'quarter map' step shown in Figure 8.

The results of the arrangement step is shown in Figure 9, which shows the `fragment_unique_values` generated from the `arrange.py` script for the equivalent information shown in the CANARI online documentation in Figure 10.

2.5.2 JDMA 'external' IDs

Ongoing work will see the data on the JDMA archive moved to the Near-Line Data Store (NLDS).[9] To assist in this process, the elastic tape 'external IDs' are also required. These numbers are found from the `jdma batch` command as shown in Listing 2):

[7] Note that this is the first of these, the *second* is named `fragment_uris_1` and so forth.
[8] see https://ncas-cms.github.io/canari/
[9] https://help.jasmin.ac.uk/docs/short-term-project-storage/nlds/

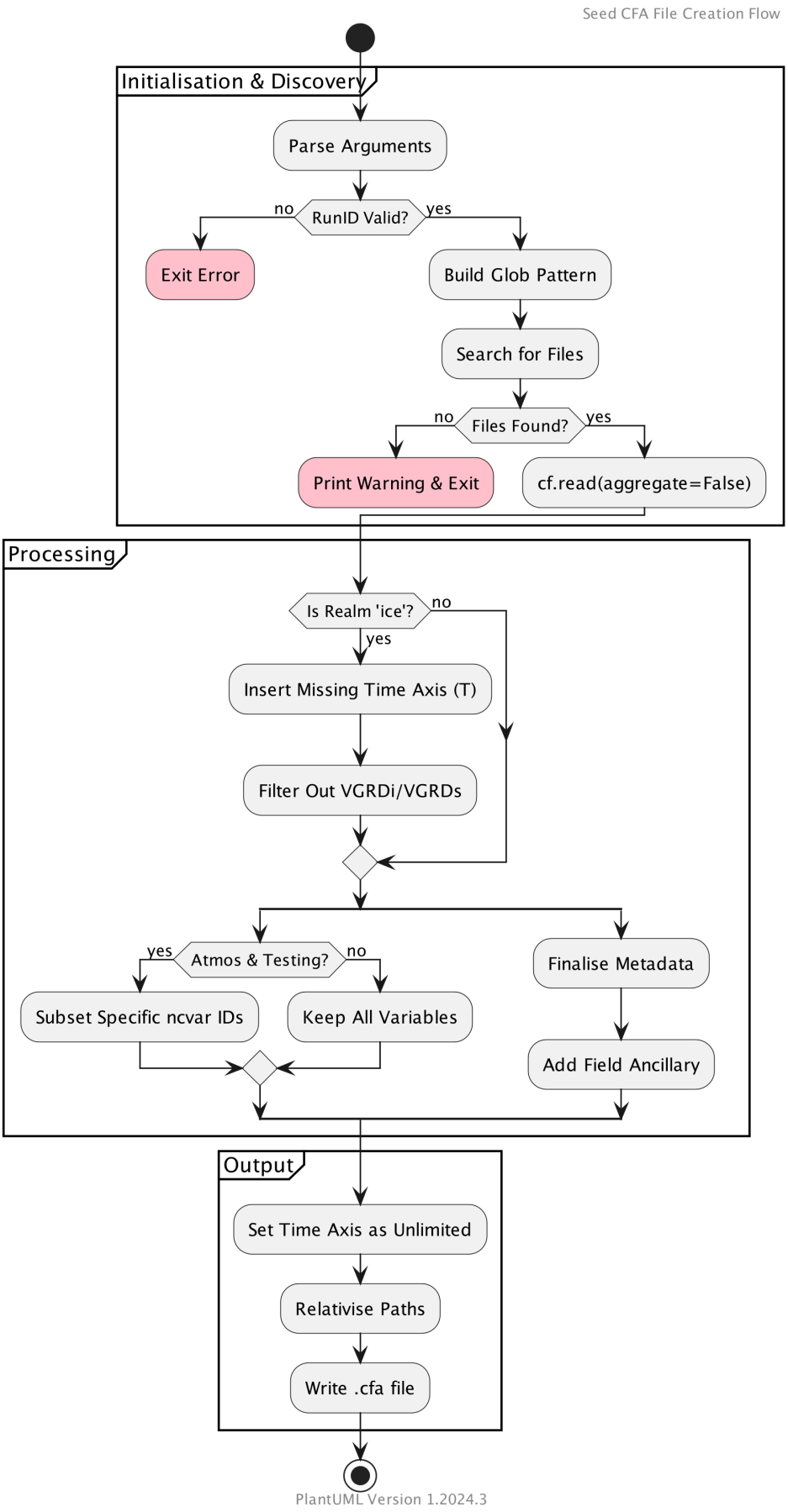

Figure 5: Algorithm for the `seed.py` file, prepared with `PlantUML` [Roques, 2026].

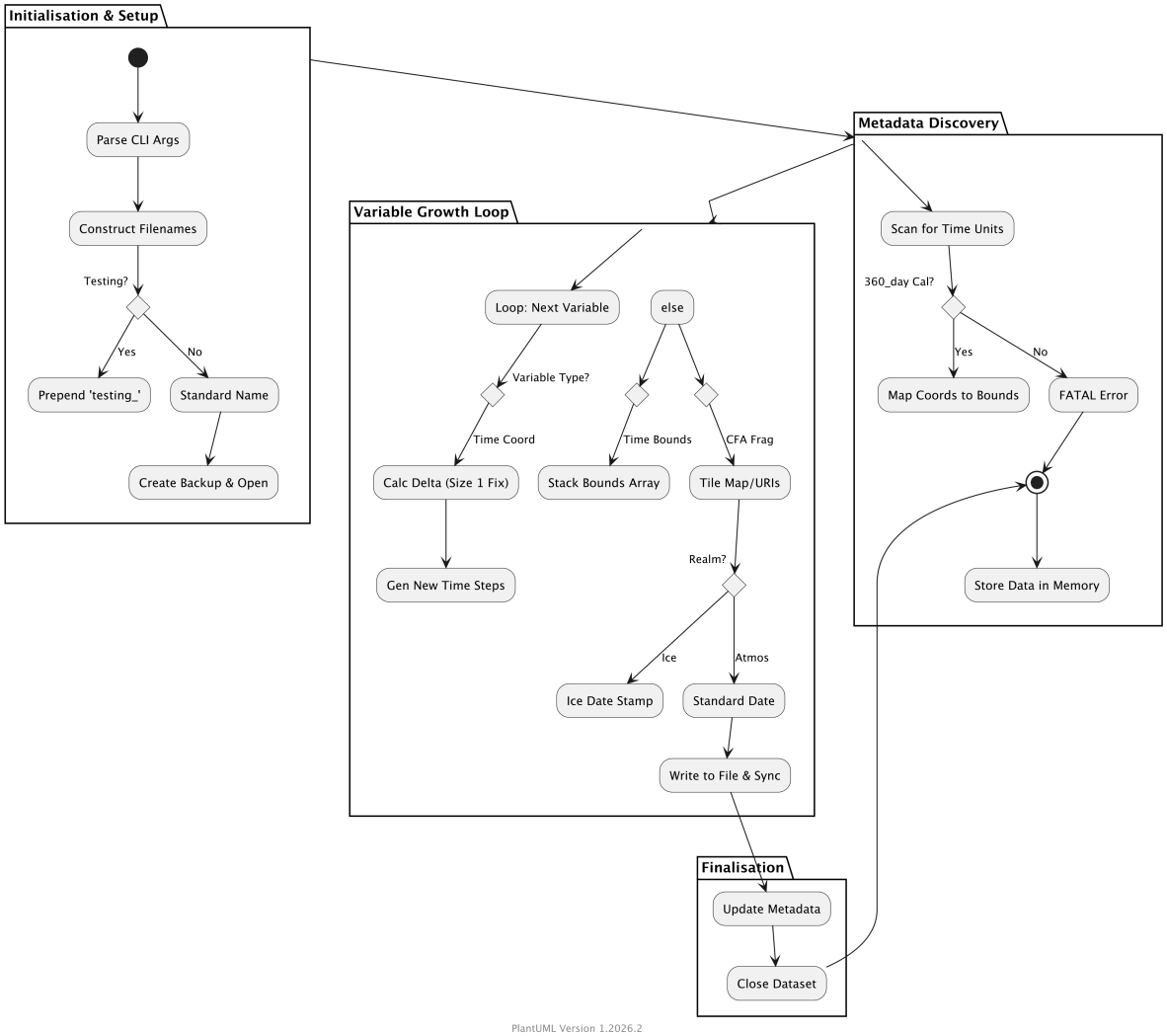

Figure 6: Algorithm for the `grow.py` file.

```
fragment_uris =
 "de814a_1_6hr_u_pt_cordex__201501-201501.nc",
 "de814a_1_6hr_u_pt_cordex__201502-201502.nc",
 "de814a_1_6hr_u_pt_cordex__201503-201503.nc",
 "de814a_1_6hr_u_pt_cordex__201504-201504.nc",
 "de814a_1_6hr_u_pt_cordex__201505-201505.nc",
 "de814a_1_6hr_u_pt_cordex__201506-201506.nc",
 "de814a_1_6hr_u_pt_cordex__201507-201507.nc",
 "de814a_1_6hr_u_pt_cordex__201508-201508.nc",
 "de814a_1_6hr_u_pt_cordex__201509-201509.nc",
 "de814a_1_6hr_u_pt_cordex__201510-201510.nc",
 "de814a_1_6hr_u_pt_cordex__201511-201511.nc",
 "de814a_1_6hr_u_pt_cordex__201512-201512.nc",
 "de814a_1_6hr_u_pt_cordex__201601-201601.nc",
 "de814a_1_6hr_u_pt_cordex__201602-201602.nc",
```

Figure 7: Example selection (first 14 months) `fragment_uris` of the grown atmosphere CFA file for the first SSP3-7.0 ensemble member.

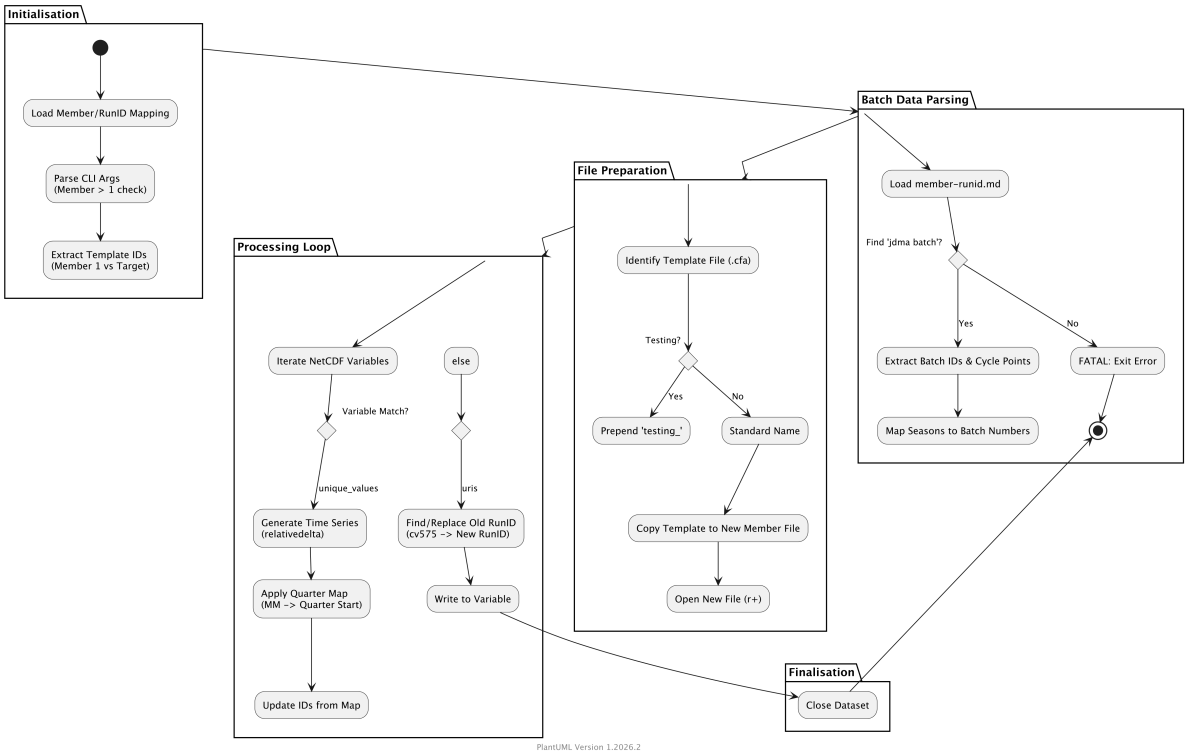

Figure 8: Algorithm for the `arrange.py` file.

```
fragment_unique_values = "18164", "18164", "18164", "18171", "18171",
    "18171", "18178", "18178", "18178", "18185", "18185", "18185", "18266",
    "18266", "18266", "18313", "18313", "18313", "18335", "18335", "18335",
    "18361", "18361", "18361", "18372", "18372", "18372", "18377", "18377",
    "18377", "18392", "18392", "18392", "18417", "18417", "18417", "18438",
    "18438", "18438", "18445", "18445", "18445", "18479", "18479", "18479",
    "18502", "18502", "18502", "18676", "18676", "18676", "18684", "18684",
    "18684", "18686", "18686", "18686", "18689", "18689", "18689", "18694",
    "18694", "18694", "18700", "18700", "18700", "18705", "18705", "18705",
    "18720", "18720", "18720", "18732", "18732", "18732", "18759", "18759",
    "18759", "18774", "18774", "18774", "18792", "18792", "18792", "18816",
    "18816", "18816", "18826", "18826", "18826", "18838", "18838", "18838",
```

Figure 9: The `fragment_unique_values` variable for the first several years of the simulation `u-de814`.

```
$ jdma batch -f workspace -w canari | grep u-de814 | grep -viE "deleted|failed" | cat -n
     1     18164 reinhard         canari           u-de814/20150101 elastictape      2024-03-25 10:28  ON_STORAGE
     2     18171 reinhard         canari           u-de814/20150401 elastictape      2024-03-25 10:33  ON_STORAGE
     3     18178 reinhard         canari           u-de814/20150701 elastictape      2024-03-25 10:38  ON_STORAGE
     4     18185 reinhard         canari           u-de814/20151001 elastictape      2024-03-25 10:43  ON_STORAGE
     5     18266 reinhard         canari           u-de814/20160101 elastictape      2024-03-26 04:08  ON_STORAGE
     6     18313 reinhard         canari           u-de814/20160401 elastictape      2024-03-26 15:15  ON_STORAGE
     7     18335 reinhard         canari           u-de814/20160701 elastictape      2024-03-27 04:37  ON_STORAGE
     8     18361 reinhard         canari           u-de814/20161001 elastictape      2024-03-27 12:49  ON_STORAGE
     9     18372 reinhard         canari           u-de814/20170101 elastictape      2024-03-27 15:35  ON_STORAGE
    10     18377 reinhard         canari           u-de814/20170401 elastictape      2024-03-27 18:47  ON_STORAGE
    11     18392 reinhard         canari           u-de814/20170701 elastictape      2024-03-28 03:28  ON_STORAGE
    12     18417 reinhard         canari           u-de814/20171001 elastictape      2024-03-28 12:29  ON_STORAGE
    13     18438 reinhard         canari           u-de814/20180101 elastictape      2024-03-28 16:31  ON_STORAGE
    14     18445 reinhard         canari           u-de814/20180401 elastictape      2024-03-28 19:28  ON_STORAGE
    15     18479 reinhard         canari           u-de814/20180701 elastictape      2024-03-29 23:20  ON_STORAGE
    16     18502 reinhard         canari           u-de814/20181001 elastictape      2024-03-30 07:07  ON_STORAGE
    17     18676 reinhard         canari           u-de814/20190101 elastictape      2024-04-02 15:27  ON_STORAGE
    18     18684 reinhard         canari           u-de814/20190401 elastictape      2024-04-02 20:23  ON_STORAGE
    19     18686 reinhard         canari           u-de814/20190701 elastictape      2024-04-02 22:24  ON_STORAGE
    20     18689 reinhard         canari           u-de814/20191001 elastictape      2024-04-02 23:59  ON_STORAGE
    21     18694 reinhard         canari           u-de814/20200101 elastictape      2024-04-03 00:54  ON_STORAGE
    22     18700 reinhard         canari           u-de814/20200401 elastictape      2024-04-03 01:50  ON_STORAGE
    23     18705 reinhard         canari           u-de814/20200701 elastictape      2024-04-03 02:50  ON_STORAGE
    24     18720 reinhard         canari           u-de814/20201001 elastictape      2024-04-03 06:26  ON_STORAGE
    25     18732 reinhard         canari           u-de814/20210101 elastictape      2024-04-03 14:49  ON_STORAGE
    26     18759 reinhard         canari           u-de814/20210401 elastictape      2024-04-04 07:01  ON_STORAGE
    27     18774 reinhard         canari           u-de814/20210701 elastictape      2024-04-04 14:43  ON_STORAGE
    28     18792 reinhard         canari           u-de814/20211001 elastictape      2024-04-04 22:04  ON_STORAGE
    29     18816 reinhard         canari           u-de814/20220101 elastictape      2024-04-05 06:25  ON_STORAGE
    30     18826 reinhard         canari           u-de814/20220401 elastictape      2024-04-05 10:11  ON_STORAGE
    31     18838 reinhard         canari           u-de814/20220701 elastictape      2024-04-05 14:47  ON_STORAGE
```

Figure 10: The JDMA batch numbers variable for the first several years of the simulation `u-de814`. The period shown is the same as shown in Figure 9.

Listing 2: `jdma batch` output

```
jdma batch 3203
    Batch for user: pug123
    Batch id     : 3203 <--- Needed in CFA files!
    Workspace    : canari
    Batch label  : u-cv575/19500101T0000Z
    Ex. storage  : elastictape
    Date         : 2023-03-22 15:44
    External id  : 33076 <--- Needed in CFA files!
    Stage        : ON_STORAGE
```

The final format of the `fragment_unique_values` strings is, e.g., `3203_33076`, i.e. `[batch_ID]_[external_ID]`.

3 Example usage and validation

Here we check that the output CFA files do indeed match the expected structure imposed on them by the method outlined above.

3.1 One day of 3-hourly, 1.5 m air temperature

Figure 11 shows code from a Jupyter notebook where the resulting CFA file for member 1 of the historical ensemble is opened using `cf.read`. After this, the 1.5 m temperature is read. This diagnostic appears many times in the output (on different time and spatial domains) and so we have then further concentrated on 3-hourly values, resulting in a single field. We then choose a random day in the 1950-2014 time series and examine the properties of the resulting 8 fields.

For this example, the date is 12/11/2004 and this is correctly identified with the file `db301a_13_3hr_pt__200411-200411.nc`. The field's ancillary data are the JDMA batches which in this case are all the same since we are only looking at one day.

The validation stage is to interrogate the data in the CANARI GitHub repository, which is itself the raw output of the `jdma batch` command, an example of which is shown in Figure 2. The final lines of output shown in Figure 11 show that on the 220th line of the relevant markdown file, the JDMA batch is 17007, matching the ancillary data values from the CFA file. Additionally, `jdma batch 17007|grep -i external` gives `External id :  54049` as expected.

3.2 Extension across batches

Here we examine an analogous situation to that in § 3.1 but where the period in question extends over multiple batches, see Figure 12.

The code shown in Figure 12 is slightly modified compared to Figure 11 in that the initial month selection is limited to January-September (inclusive) to ensure that when a 3-month 'time delta' is added to the randomly chosen date, the year does not need to be changed to reflect this. This is purely for illustrative convenience and is not a limitation of the code.

Figure 12 gives the randomly assigned initial date as 21/1/1998 and extends to 21/4/1998 giving the following list of files (the highlighted file indicates its presence in a different JDMA batch to the others):

1. `db301a_13_3hr_pt__199801-199801.nc`
2. `db301a_13_3hr_pt__199802-199802.nc`
3. `db301a_13_3hr_pt__199803-199803.nc`

```python
f = cf.read(
    glob.glob("./CF-1.13_grown_CANARI_27_db301_atmos_65years_batches.cfa"),
)

y = np.random.randint(1950, 2014)
m = np.random.randint(1, 12)
d = np.random.randint(1, 30)
print(f"The date is {y:04d}-{m:02d}-{d:02d}")

target_field = f.select_by_property(
    standard_name="surface_temperature", interval_operation="3 h"
)[0].subspace(T=cf.wi(cf.dt(y, m, d, 0, 0), cf.dt(y, m, d, 23, 59)))

pprint("File name or names...")
pprint(target_field.data.get_filenames())
pprint("The unique ancillary batch or batches are...")
np.unique(
    target_field.constructs.filter(
        filter_by_property={"long_name": re.compile(".*jdma.*", re.I)}
    )
    .value()
    .array
).tolist()
```

```
The date is 2004-11-12
'File name or names...'
{'db301a_13_3hr_pt__200411-200411.nc'}
'The unique ancillary batch or batches are...'
['17007_54049']
```

```python
# for _m in [m, m + 3]:

m_formatted = f"{m:02d}"
search_pattern = str(y) + quarter_map[m_formatted]

for file in glob.glob("/home/users/jonnyhtw/canari/docs/hist2/*db301*"):
    for line in open(file):
        if search_pattern in line:
            print(line)
```

```
220     17007 jrobson          canari          u-db301/20041001 elastictape      2024-02-23 16:55  ON_STORAGE
```

Figure 11: Validation of one day of 3-hourly 1.5 m air temperature output; comparison of CFA file resulting from this work and the raw JDMA batch information.

```python
f = cf.read(
    glob.glob("./CF-1.13_grown_CANARI_27_db301_atmos_65years_batches.cfa"),
)

y = np.random.randint(1950, 2014)
m = np.random.randint(1, 9)
d = np.random.randint(1, 30)
print(f"The date is {y:04d}-{m:02d}-{d:02d}")

target_field = f.select_by_property(
    standard_name="surface_temperature", interval_operation="3 h"
)[0].subspace(T=cf.wi(cf.dt(y, m, d, 0, 0), cf.dt(y, m + 3, d, 23, 59)))

pprint("File name or names...")
pprint(target_field.data.get_filenames())
pprint("The unique ancillary batch or batches are...")
np.unique(
    target_field.constructs.filter(
        filter_by_property={"long_name": re.compile(".*jdma.*", re.I)}
    )
    .value()
    .array
).tolist()
```

```
The date is 1998-01-21
'File name or names...'
{'db301a_13_3hr_pt__199801-199801.nc',
 'db301a_13_3hr_pt__199802-199802.nc',
 'db301a_13_3hr_pt__199803-199803.nc',
 'db301a_13_3hr_pt__199804-199804.nc'}
'The unique ancillary batch or batches are...'
['16552_52832', '16557_52853']
```

```python
for _m in [m, m + 3]:

    m_formatted = f"{_m:02d}"
    search_pattern = str(y) + quarter_map[m_formatted]

    for file in glob.glob("/home/users/jonnyhtw/canari/docs/hist2/*db301*"):
        for line in open(file):
            if search_pattern in line:
                print(line)
```

```
193     16552 jrobson          canari          u-db301/19980101 elastictape      2024-02-17 01:24  ON_STORAGE

194     16557 jrobson          canari          u-db301/19980401 elastictape      2024-02-17 04:29  ON_STORAGE
```

Figure 12: As for Figure 11 but over a period of 3 months, i.e. traversing two JDMA batches.

4. `db301a_13_3hr_pt__199804-199804.nc`

The unique JDMA batch numbers are given as 16552 and 16557. Again these agree with the 'raw' `jdma batch` output.

For the external IDs, once again they agree (Figure 12).

- `jdma batch 16552|grep -i external External id :  52832`
- `jdma batch 16557|grep -i external External id :  52853`

4 Conclusions

This work has expanded on previous work to catalogue the variables in the CANARI historical and SSP3-7.0 scenario ensembles. This has been done using the aggregation capabilities of the `cf` Python package [Hassell et al., 2017], termed 'CFA'.

This process involves three discrete steps; seeding, growth and arrangement. The first of these generates the template (i.e. 'seed') for the first month of a reference simulation, the second grows it out to the length of the simulation inferring the already-standardised filenames, and the third assigns the correct tape archive 'batch' numbers thus enabling later file discovery ('arrangement').

The work has been version controlled using Git and is broadly applicable to similarly-standardised datasets of arbitrary size.

The largest time resource is taken up by the primary seeding step. This can be seen in Appendix A which shows that the generation of the seed file for the atmosphere realm of the historical scenario takes approximately 1 hour, the growth step take 16 minutes and arrangement 6 seconds.

That said, the seeding only needs to be run once for each data instance; e.g. atmosphere data for a particular climate scenario. In this case there was a small change to the diagnostic set in ensemble members 13-40 which necessitated the generation of a different seed, but this only adds $\mathcal{O}(1 \text{ hour})$ to the overall computation time since a new seed file must be created.

These scripts – although predicated on the CANARI ensemble and pre-formatted filenames – could straightforwardly be edited and used by other large datasets.

Appendix A Benchmarking

This section compares the time taken in each of the seed, grow and arrange steps for the atmosphere realm of `u-cy537`, the 13th member of the historical ensemble; see Table 1.

A.1 Seed

Listing 3

Listing 3: `seed.py`

```
time python seed.py --runid     cy537\
                    --member    13\
                    --realm     atmos\
                    --data_path $HOME/archive/CANARI/u-cy537/19500101T0000Z/\
                    --startyear 1950
```

Table 2: Timings for arrange, historical, atmosphere, using the output from the `time` Linux command. The 'real' time is the 'Elapsed real time (in seconds)', 'user' is 'Total number of CPU-seconds that the process spent in user mode.' and 'sys' is 'Total number of CPU-seconds that the process spent in kernel mode.' where the definitions are from the `man time` Linux command.

Measure	time
real	59 m 15 s
user	55 m 18 s
sys	2 m 37 s

A.2 Grow

As for § A.1 for the growth step. Code usage in Listing 4.

Listing 4: `grow.py`

```
time python grow.py --runid   cy537\
                    --member  13\
                    --realm   atmos\
                    --n_years 65
```

Table 3: Timings for grow, historical, atmosphere, using the output from the `time` Linux command. The 'real' time is the 'Elapsed real time (in seconds)', 'user' is 'Total number of CPU-seconds that the process spent in user mode.' and 'sys' is 'Total number of CPU-seconds that the process spent in kernel mode.' where the definitions are from the `man time` Linux command.

Measure	time
real	16 m 16 s
user	15 m 16 s
sys	0 m 27 s

A.3 Arrange

As for § A.1 for the arrange step. Code usage shown in Listing 5.

Listing 5: `arrange.py`

```
time python arrange.py --member   13\
                       --realm    atmos\
                       --n_years  65\
                       --scenario hist2
```

A.4 Comparison of time taken in seed, grow and arrange steps

Figure 13 shows the 'real' time taken as shown in Tables 2, 3 and 4.

Appendix B Runtime instructions

This section shows the command line 'doc string' help available for the seed, grow and arrange steps.

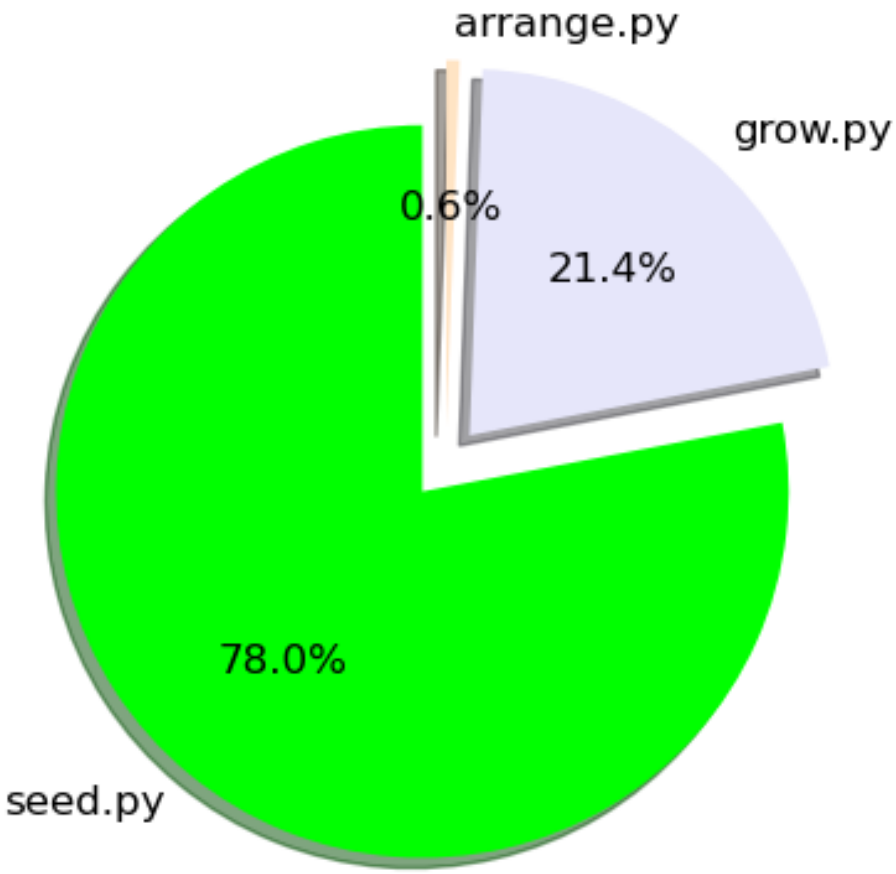

Figure 13: Pie graph representation of the ‘real’ times from Tables 2, 3 and 4.

Table 4: Timings for arrange, historical, atmosphere, using the output from the `time` Linux command. The 'real' time is the 'Elapsed real time (in seconds)', 'user' is 'Total number of CPU-seconds that the process spent in user mode.' and 'sys' is 'Total number of CPU-seconds that the process spent in kernel mode.' where the definitions are from the `man time` Linux command.

Measure	time
real	0 m 26 s
user	0 m 25 s
sys	0 m 2 s

B.1 Seed

Listing 6.

Listing 6: Output of python seed.py -h

```
usage: seed.py [-h] --runid RUNID --realm {atmos,ocean,ice} --member MEMBER [--testing
    {yes,no}] --startyear STARTYEAR
               --data_path DATA_PATH

Initializes CFA files from source data.

options:
  -h, --help            show this help message and exit
  --runid RUNID         Run ID in format letter-letter-number-number-number
  --realm {atmos,ocean,ice}
                        The climate realm
  --member, -m MEMBER   The ensemble member number
  --testing, -t {yes,no}
                        Is this a test run? (default: no)
  --startyear STARTYEAR
                        Starting year of the simulation.
  --data_path, -d DATA_PATH
                        Path to the source data directory

(cfa_env) sci-vm-01|Mon Mar 23|16:51:12 GMT|cfa_scripts|[main]>
```

B.2 Grow

Listing 7.

Listing 7: Output of python grow.py -h

```
usage: grow.py [-h] --runid RUNID --realm {atmos,ocean,ice} --member MEMBER [--testing
    {yes,no}] --n_years N_YEARS

Appends data to existing CFA files.

options:
  -h, --help            show this help message and exit
  --runid RUNID
  --realm {atmos,ocean,ice}
  --member, -m MEMBER
  --testing, -t {yes,no}
                        Is this a test run? (default: no)
  --n_years, -ny N_YEARS
```

```
(cfa_env) sci-vm-01|Mon Mar 23|16:52:48 GMT|cfa_scripts|[main]>
```

B.3 Arrange

Listing 8.

Listing 8: Output of python arrange.py -h

```
usage: arrange.py [-h] --member MEMBER --realm {atmos,ocean,ice} [--testing {yes,no}]
    --n_years N_YEARS --scenario {hist2,ssp370}

Adds in JDMA batch numbers.

options:
  -h, --help            show this help message and exit
  --member, -m MEMBER   Ensemble member.
  --realm {atmos,ocean,ice}
  --testing, -t {yes,no}
                        Is this a test run? (default: no)
  --n_years, -ny N_YEARS
  --scenario, -s {hist2,ssp370}

(cfa_env) sci-vm-01|Mon Mar 23|16:54:10 GMT|cfa_scripts|[main]>
```

Appendix C Priority variables

C.1 Background

The entirety of this report has dealt with the CF aggregration of the entire CANARI output as stored (currently, May 2026) on the JDMA system. For many applications however, only a subset of data is required and this is referred to as the 'priority variables' – PVs – set. Detailed information regarding the variables themselves can be found on the relevant NCAS-CMS GitHub page.

C.2 Algorithm

For the priority variables, all data is accessible 'live' on the 'canari' Group Workspace on JASMIN at '/gws/ssde/j25b/canari/large-ensemble/priority-variables'. In addition, the PVs are pre-separated into single variable files, for example (see Figure 15):

- `cv575a_1_1hr_m01s05i216.nc`, for the atmosphere.
  - Surface temperature after timestep [°K], details of variables can be found on the UM STASH browser.
- `cv575o_1_day__grid_T_tos.nc`, for the ocean.
  - Sea surface temperature [°C].
- `cv575i_1_day_aice.nc`, for the sea ice.
  - Ice area (aggregate) $\in [0, 1]$.

Because of the availability of the PVs on disk, the process for obtaining the CFA files representing each realm is greatly simplified and is performed by a single Python script,

`priority-variables.py`. This script reads in all the data for each member of the historical and SSP3-7.0 ensembles and writes out one CFA files for each variable/scenario/member combination. The `priority-variables` Python script which performs this is illustrated using `PlantUML` in Figure 14.

Listing 9: Output of python priority-variables.py --help

```
python priority-variables.py --help
Usage: priority-variables.py [OPTIONS]

  Create seed CFA file for CANARI priority variables.

Options:
  --realm [ATM|OCN|CICE]     The climate realm  [required]
  --verbose INTEGER RANGE    Verbosity level of cf.read  [-1<=x<=3]
  --scenario [HIST2|SSP370]  The scenario  [required]
  -m, --member INTEGER       The ensemble member number  [required]
  -d, --data_path PATH       Path to the source data directory  [required]
  --help                     Show this message and exit.
(cfa_env) sci-vm-01|Wed May 27|15:44:32 BST|priority-variables|[priority-variables]>
```

The code for generating Figure 15 is shown in Listing 10.

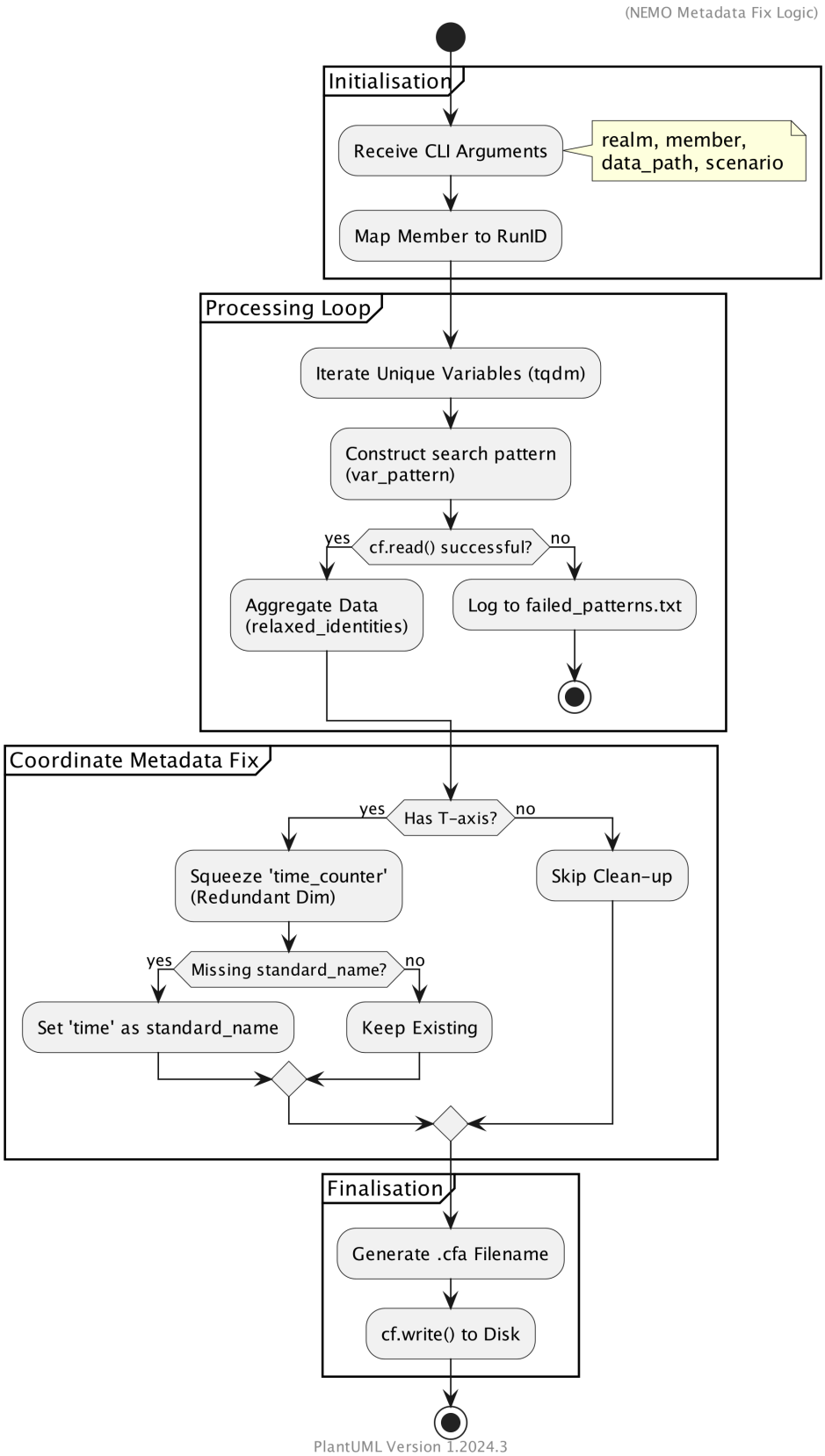

Figure 14: Algorithm for the `priority-files.py` file, Figure prepared using `PlantUML`.

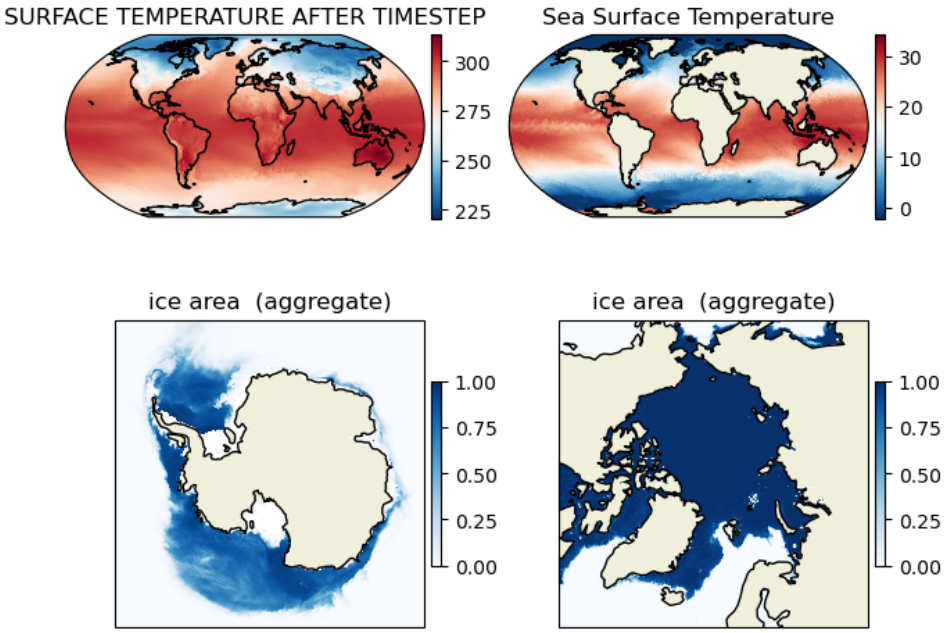

Figure 15: Example surface air temperature after timestep (in Kelvin), sea surface temperature (in Celsius), and sea ice area (aggregate) from ensemble member 1 of the historical ensemble.

Listing 10: Code to generate Figure 15.

```
import matplotlib.pyplot as plt
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import cf

files = [
    "./CF-1.13_seed_CANARI_1_cv575_ATM_1_mon_m01s00i024_3.cfa",
    "./CF-1.13_seed_CANARI_1_cv575_OCN_day__grid_T_tos.cfa",
    "CF-1.13_seed_CANARI_1_cv575_CICE_day_aice.cfa",
] + [
    "CF-1.13_seed_CANARI_1_cv575_CICE_day_aice.cfa",
]

fig, axes = plt.subplots(
    nrows=2,
    ncols=2,
    layout="constrained",
)

if len(files) == 1:
    axes = [axes]
else:
    axes = axes.flatten()

cmaps = ["RdBu_r"] * 2 + ["Blues"] * 2

for i in range(len(files)):
    old_ax = axes[i]
    pos = old_ax.get_subplotspec()
    old_ax.remove()

    if i < 2:
        proj = ccrs.Robinson()
    elif i == 2:
        proj = ccrs.SouthPolarStereo()
    elif i == 3:
        proj = ccrs.NorthPolarStereo()

    ax = fig.add_subplot(pos, projection=proj)

    fields = cf.read(files[i])
    f = fields[0][0]

    mesh = ax.pcolormesh(
        f.construct("X").data.array,
        f.construct("Y").data.array,
        f.data.array.squeeze(),
        transform=ccrs.PlateCarree(),
        shading="auto",
        cmap=cmaps[i],
        zorder=0,
    )

    ax.coastlines(resolution="110m", linewidth=1)

    if i == 2:
        ax.set_extent([-180, 180, -90 * (i == 2), -60 * (i == 2)], ccrs.PlateCarree())
```

```
    elif i == 3:
        ax.set_extent([-180, 180, 60, 90], ccrs.PlateCarree())
    else:
        ax.set_global()

    if i > 0:
        ax.add_feature(cfeature.LAND, zorder=1)

    plt.colorbar(mesh, ax=ax, orientation="vertical", pad=0.02, shrink=0.6)

    title_text = getattr(f, "long_name", f"File {i}")
    ax.set_title(title_text)

plt.show();
```

C.3 Discovery and generation of single-variable files

The full CANARI data set is held on the JDMA system – see page 4 – but to retrieve individual variables there is a 2 step process:

1. Retrieve monthly data from JDMA for the years(s) required (see Figure 16).
2. Extract the required data and write to single-variable files (see Figure 17).

C.3.1 Retrieval

Figure 16 shows this graphically and Listing 11 gives the code implementation.

Listing 11: Output of python jdma_get_monthly.py --help

```
> python jdma_get_monthly.py --help
usage: jdma_get_monthly.py [-h] [--start START] [--end END] [--cycle CYCLE] [--
    filelist] [--ens ENS] suite

positional arguments:
  suite          Suite ID to extract data from

options:
  -h, --help     show this help message and exit
  --start START  Year from which to start extracting data from
  --end END      Year to extract data up to (included)
  --cycle CYCLE  Cycle (YYYYMM) for which to extract data for. Cannot be used in
    combination with --start/--end
  --filelist     Create file(s) containing a list of files that would be extracted
  --ens ENS      Ensemble number of suite
(cfa_env) sci-vm-01|Tue May 26|15:27:35 BST|fix_corrupted|[priority-variables]>
```

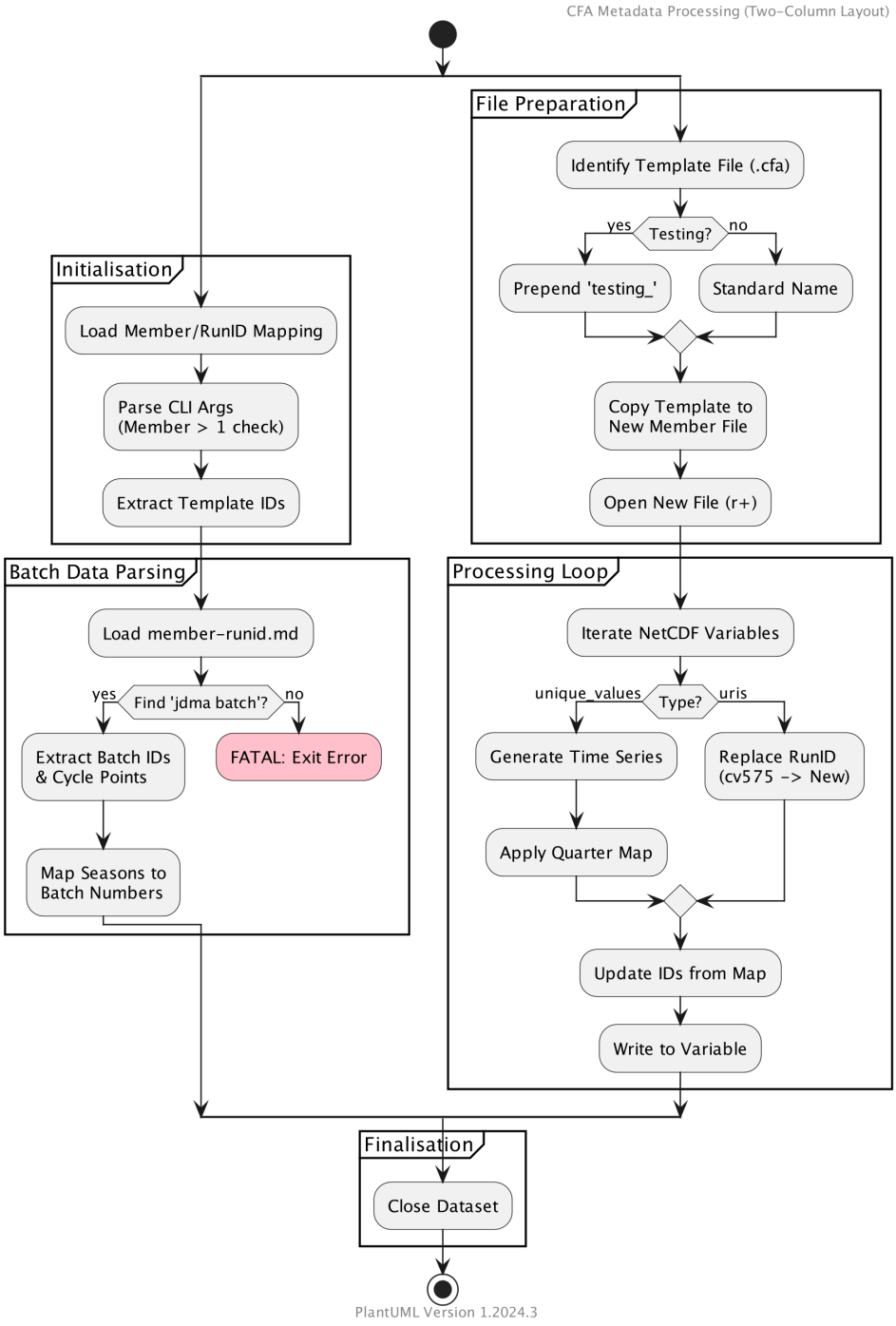

Figure 16: Algorithm for the `jdma_get_monthly.py` file, Figure prepared using `PlantUML`.

C.3.2 Extraction to yearly

Figure 17 shows this graphically and Listing 12 gives the code implementation.

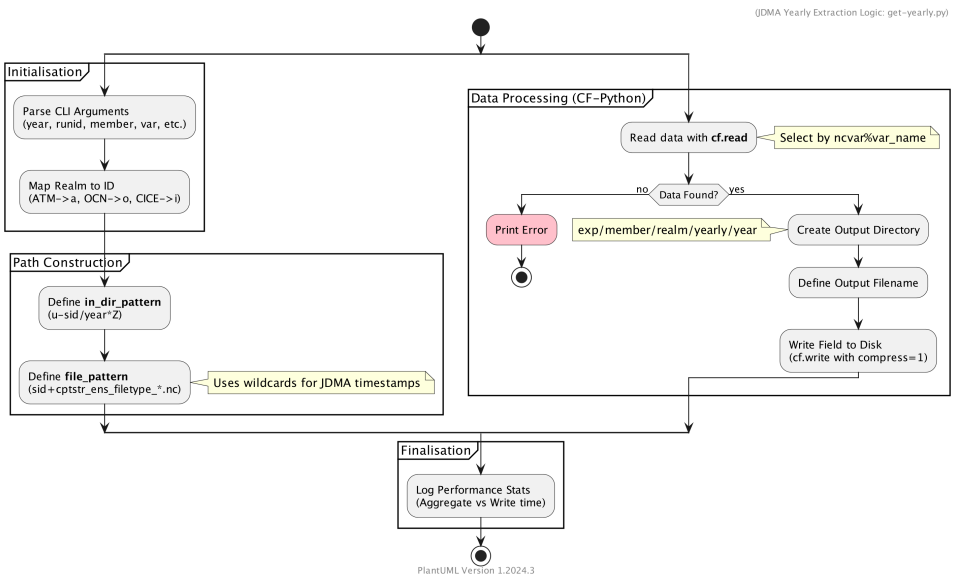

Figure 17: Algorithm for the `get-yearly.py` file, Figure prepared using `PlantUML`.

Listing 12: Output of python jget-yearly.py --help

```
> python get-yearly.py --help
usage: get-yearly.py [-h] --year YEAR --runid RUNID --member MEMBER --scenario
    SCENARIO --realm {ATM,OCN,CICE}
                     --filetype FILETYPE --var VAR

Extract single-year variables from JDMA-retrieved data.

options:
  -h, --help            show this help message and exit
  --year YEAR           Year to process (e.g. 1975)
  --runid RUNID         Run ID without the u- prefix (e.g. cz649)
  --member MEMBER       Ensemble member number
  --scenario SCENARIO   Scenario (e.g. HIST2)
  --realm {ATM,OCN,CICE}
                        Realm
  --filetype FILETYPE   Type (e.g. mon__diaptr)
  --var VAR             Var name (e.g. zotematl)
```

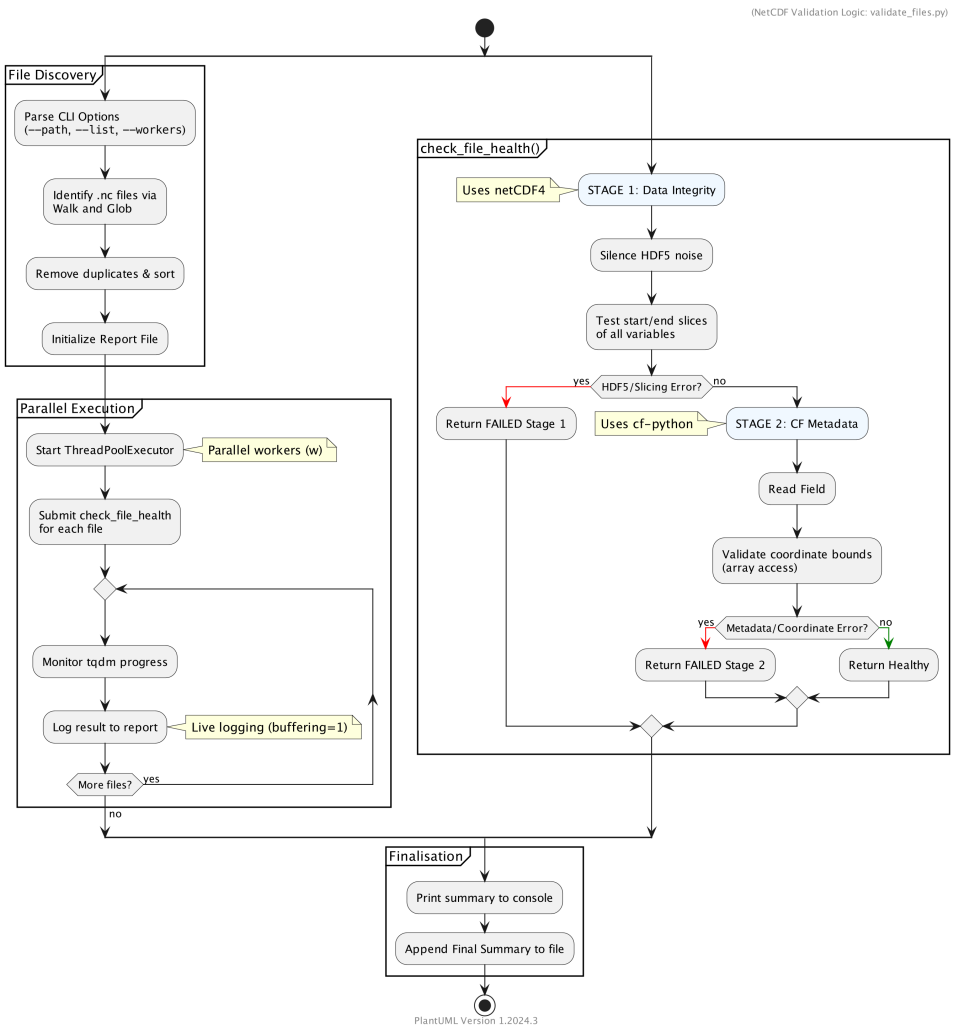

Figure 18: Algorithm for the `validate-files.py` file, Figure prepared using `PlantUML`.

C.4 Validation of corrupt priority-variable files

Figure 18 shows the validation process graphically and Listing 13 shows the code implementation.

Listing 13: Output of python validate-files.py --help

```
> python validate-files.py  --help
Usage: validate-files.py [OPTIONS]

  Scan NetCDF files and generate a full Health Report (Live Logging).

Options:
  -p, --path TEXT         Path to scan.
  -l, --list PATH         Text file with paths.
  -w, --workers INTEGER   Number of parallel workers.
  -o, --out TEXT          Output report file.
  --help                  Show this message and exit.
(cfa_env) sci-vm-01|Tue May 26|16:02:45 BST|fix_corrupted|[priority-variables]>
```

The validation process is twofold; the checking of the presence of coordinate bounds, and the fidelity of the first and last slices of each diagnostic. The choice of these two tests was heuristic in the sense that there are many different tests that could be performed to assess the quality of model data but these tests were found sufficient to detect – and subsequently fix – all corrupt

files.

Note that to enable corrupt-file discovery the `priority-variables.py` script shown in Figure 14 and Listing 9 enables the writing of Python 'glob' strings which cause the CFA generation process to fail. These lists can then be used to 'home in' on specific broken files.

References

Brian Eaton, Jonathan Gregory, Bob Drach, Karl Taylor, Steve Hankin, John Caron, Rich Signell, Phil Bentley, Greg Rappa, Heinke Höck, Alison Pamment, Martin Juckes, Martin Raspaud, Jon Blower, Randy Horne, Timothy Whiteaker, David Blodgett, Charlie Zender, Daniel Lee, David Hassell, Alan D. Snow, Tobias Kölling, Dave Allured, Aleksandar Jelenak, Anders Meier Soerensen, Lucile Gaultier, Sylvain Herlédan, Fernando Manzano, Lars Bärring, Christopher Barker, Sadie L. Bartholomew, Thomas Lavergne, Bryan Lawrence, Neil Massey, Antonio S. Cofiño, Seth McGinnis, and Patrick Van Laake. NetCDF Climate and Forecast (CF) Metadata Conventions, December 2025. URL https://doi.org/10.5281/zenodo.17801666.

D. Hassell, J. Gregory, J. Blower, B. N. Lawrence, and K. E. Taylor. A data model of the climate and forecast metadata conventions (cf-1.6) with a software implementation (cf-python v2.1). *Geoscientific Model Development*, 10(12):4619–4646, 2017. doi: 10.5194/gmd-10-4619-2017. URL https://gmd.copernicus.org/articles/10/4619/2017/.

International Organization for Standardization. Date and time – Representations for information interchange – Part 1: Basic rules. ISO 8601-1:2019, 2019. URL https://www.iso.org/standard/70631.html. Standard.

IPCC. Summary for policymakers. In *Climate Change 2023: Synthesis Report. Contribution of Working Groups I, II and III to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change*. IPCC, Geneva, Switzerland, 2023. doi: 10.59327/IPCC/AR6-9789291691647.001.

Hilary J. Oliver, Matthew Shin, and Oliver Sanders. Cylc: A workflow engine for cycling systems. *Journal of Open Source Software*, 3(27):737, 2018. doi: 10.21105/joss.00737. URL https://doi.org/10.21105/joss.00737.

Arnaud Roques. Plantuml: Open-source tool that uses simple textual description to draw uml diagrams, 2026. URL https://plantuml.com/. Accessed: 2026-03-19.

R. Schiemann et al. The CANARI HadGEM3 Large Ensemble: Design and evaluation of historical simulations. in prep., 2026.

K. D. Williams, D. Copsey, E. W. Blockley, A. Bodas-Salcedo, D. Calvert, R. Comer, P. Davis, T. Graham, H. T. Hewitt, R. Hill, P. Hyder, S. Ineson, T. C. Johns, A. B. Keen, R. W. Lee, A. Megann, S. F. Milton, J. G. L. Rae, M. J. Roberts, A. A. Scaife, R. Schiemann, D. Storkey, L. Thorpe, I. G. Watterson, D. N. Walters, A. West, R. A. Wood, T. Woollings, and P. K. Xavier. The Met Office global coupled model 3.0 and 3.1 (GC3.0 and GC3.1) configurations. *Journal of Advances in Modeling Earth Systems*, 10(2):357–380, 2018. doi: 10.1002/2017MS001115. URL https://doi.org/10.1002/2017MS001115.